CSI3007- Advanced Python Programming

Name: Nideesh.C.G

Reg.No: 23MIC0092

Slot: C1+TC1

Team Member: Dharma.S

Team Member Reg.No: 23MIC0104

Title: Circular E-Waste Recovery & Refurbishment Intelligence System

Project Overview

The Circular E-Waste Recovery & Refurbishment Intelligence Platform is a distributed web-based system designed to connect electronic-waste collection operations with device assessment and refurbishment workflows. It consists of two independently deployed Python applications that exchange device information and assessment results through REST APIs.

This write-up describes the project architecture and implementation context. Exact API payloads, frontend framework details, upload restrictions and production deployment health should be checked against the final source code and live environment before submission.

1. Title

Circular E-Waste Recovery & Refurbishment Intelligence Platform

A Bidirectional, API-Driven Platform for Electronic Waste Collection, Device Assessment, Refurbishment and Resource Recovery.

2. Motivation

The rapid advancement of technology has increased the consumption and replacement of electronic devices such as laptops, smartphones, computers and other electrical equipment. As devices become obsolete or damaged, electronic waste (e-waste) is generated.

Many discarded devices may still contain reusable components or may be suitable for repair and refurbishment. However, collection centres and refurbishment facilities can operate with disconnected records and workflows. This can make it difficult to track devices, evaluate recovery potential and coordinate transfers.

The Global E-waste Monitor 2024 reports that approximately 62 million tonnes of e-waste were generated globally in 2022, while 22.3% was documented as formally collected and recycled. This global challenge motivated the development of a connected digital workflow that supports device traceability, assessment and recovery-oriented decision-making.

The project aims to support circular-economy practices by making device information available across collection and refurbishment stages, helping stakeholders consider repair, reuse, component recovery and recycling pathways.

3. Existing Gap, Problem Statement and USP

3.1 Existing Gap

- Device registration, transfer management and refurbishment assessment may be maintained in separate systems.
- Information can become fragmented when collection and refurbishment facilities do not share a common workflow.
- Transfer status may not accurately reflect whether the receiving application has persisted a device record.
- Assessment outcomes may not be returned to the original collection record in a structured way.
- Supporting images and documents need a secure and consistent method of exchange.

3.2 Problem Statement

There is a need for an integrated digital platform that connects e-waste collection and refurbishment operations, enables reliable bidirectional information exchange, maintains device traceability and records assessment outcomes against the corresponding device.

3.3 Proposed Solution

Application 1 – E-Waste Collection & Recovery

- Register electronic devices and capture device specifications and condition details.
- Manage device records and transfer requests.
- Upload supporting device images and documents where supported.
- Track transfer status and receiving assessment results from the refurbishment application.

Application 2 – Circular Refurbishment Intelligence

- Receive device information from the collection application.
- Maintain incoming-device and assessment records.
- Record repairability, recovery, cost and value estimates.
- Prepare refurbishment or parts-recovery recommendations.
- Send assessment results back to the collection application.

3.4 Unique Selling Proposition (USP)

The central USP is the bidirectional integration of two independent applications through a common REST API. The system connects device transfer with assessment-result feedback.

- Bidirectional communication: device information flows from collection to refurbishment, and assessment results flow back.

- Independent applications: each application has its own backend and deployment lifecycle.
- Device traceability: device and transfer identifiers associate records across the applications.
- Assessment workflow: repairability and recovery-related values can be recorded in a structured format.
- Cloud-based file exchange: supporting file references can be shared using common cloud storage.
- Recovery-oriented workflow: assessment information can support decisions about refurbishment, parts recovery or recycling.

4. Architecture

4.1 High-Level System Architecture

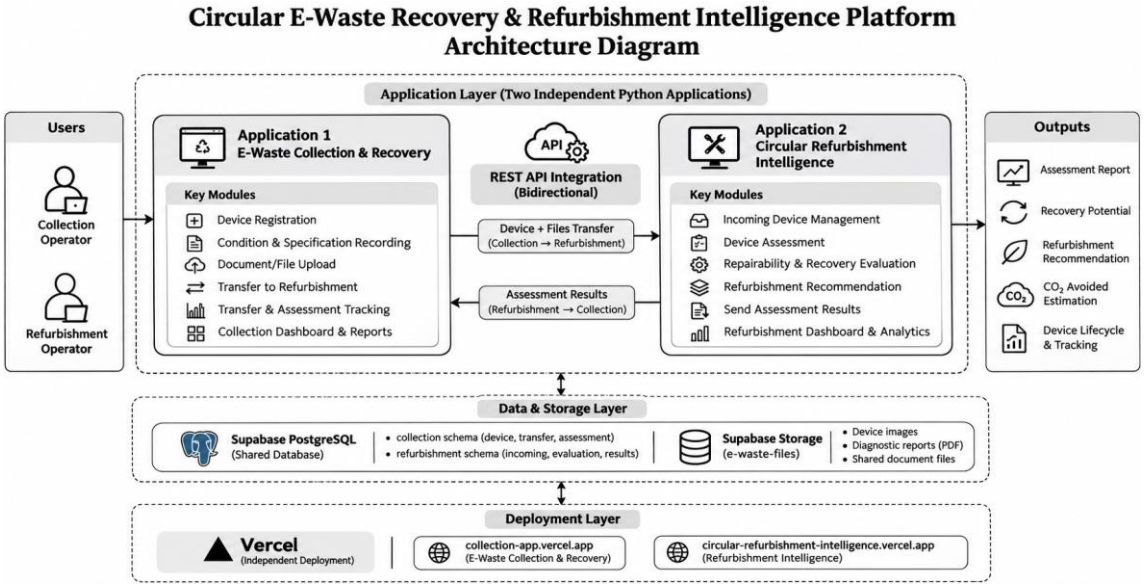


Figure 1. High-level System architecture and bidirectional data flow.

System Architecture Explanation

The Circular E-Waste Recovery & Refurbishment Intelligence Platform follows a modular, layered architecture consisting of two independent Python applications connected through a bidirectional REST API. The architecture is designed to support efficient e-waste collection, device information exchange, refurbishment assessment, data management, and recovery analysis.

1. User Layer

The platform supports two primary categories of users: Collection Operators and Refurbishment Operators. Collection Operators use the E-Waste Collection and Recovery application to register electronic devices, record their condition and specifications, upload relevant documents, and transfer device information to the refurbishment application. Refurbishment Operators use the Circular Refurbishment Intelligence application to inspect incoming devices, perform assessments, evaluate repairability, and generate refurbishment recommendations.

2. Application Layer

This layer consists of two independent Python-based applications.

- **Application 1 – E-Waste Collection & Recovery:** This application manages the initial stages of the e-waste lifecycle. Its modules include device registration, condition and specification recording, document uploads, device transfer, transfer tracking, and collection dashboard management.
- **Application 2 – Circular Refurbishment Intelligence:** This application handles the technical evaluation and recovery of collected electronic devices. It provides incoming device management, device assessment, repairability evaluation, recovery potential analysis, refurbishment recommendations, and assessment result generation.

Both applications operate independently while communicating through a common REST API.

3. Bidirectional REST API Integration Layer

The REST API acts as the communication bridge between the two applications and enables bidirectional data exchange.

- **Collection to Refurbishment:** Device details, specifications, images, and supporting documents are transferred from the E-Waste Collection application to the Refurbishment Intelligence application.
- **Refurbishment to Collection:** After evaluating a device, the refurbishment application sends assessment results, repairability information, recovery recommendations, and related outcomes back to the collection application.

This integration ensures that both applications remain synchronized and that information flows throughout the device recovery process.

4. Data and Storage Layer

The platform uses Supa base PostgreSQL as its database management system for storing structured application data. The database maintains collection-related information, incoming device records, assessment details, refurbishment evaluations, and recovery results.

Supa base Storage is used to manage supporting files such as device images, diagnostic reports in PDF format, and other shared documents. This separation of structured data and file storage supports organized data management and efficient retrieval.

5. Deployment Layer

Both applications are deployed independently using Vercel, making them accessible through separate web addresses. The E-Waste Collection application and Circular Refurbishment Intelligence application can be accessed online without requiring local installation. Independent deployment also allows each application to be maintained and updated separately while preserving communication through the REST API.

6. Output Layer

The platform generates meaningful outputs that support e-waste recovery decisions. These include device assessment reports, recovery potential information, refurbishment recommendations, estimated CO_2 emissions avoided, and device lifecycle tracking. These outputs help operators understand the condition of collected devices and determine suitable recovery pathways.

Overall Architecture

The architecture establishes an integrated digital ecosystem for managing the e-waste recovery lifecycle. By combining two independent Python applications, bidirectional REST API communication, centralized data management, cloud-based file storage, and independent online deployment, the platform provides a structured approach to device collection, assessment, refurbishment, and recovery tracking.

4.2 Architecture Components

Component	Responsibility
Collection application	Device registration, condition capture, transfer initiation and receipt of assessment results.
Refurbishment application	Incoming-device management, assessment recording and recovery recommendations.
REST API layer	Structured communication between the independent applications.
Supabase PostgreSQL	Persistent relational data storage with logically separated schemas.
Supabase Storage	Cloud storage for supporting documents and device files.
API-key authentication	Authentication of server-to-server API requests.
Vercel	Independent hosting/deployment of the applications.

4.3 Database Organization

The shared Supabase PostgreSQL project is intended to maintain separate schemas for the two applications:

- collection – collection-side device, transfer and related records.
- refurbishment – incoming-device, assessment, transfer and related records.

This organization provides logical separation while allowing both applications to use a common managed database service. The exact tables and relationships should be described using the final database schema.

4.4 File Storage

Supabase Storage is used as the shared cloud file-storage layer. The project configuration identifies a private bucket named ewaste-files, with application-specific prefixes intended to separate uploaded content. File metadata or storage references are exchanged through the API rather than relying on local server files.

5. Complete Technology Stack

Layer	Technology	Purpose
Programming language	Python	Backend logic and application services.
Backend framework	Flask	Web application routes and REST API endpoints.
Frontend	HTML, CSS, JavaScript and application frontend assets	User interface and browser interactions; confirm any framework used in the final build.
ORM	SQLAlchemy	Relational database models and queries.
Database	PostgreSQL	Persistent storage for application records.
Database platform	Supabase	Managed PostgreSQL service.
File storage	Supabase Storage	Cloud storage for uploaded files.
API style	REST	Communication between the two applications.
Data interchange	JSON	Structured API requests and responses.
Database driver	Psycopg	Python connectivity to PostgreSQL.
Migrations	Alembic / Flask-Migrate	Database schema versioning and migrations.

Authentication	API-key authentication	Protects server-to-server communication.
Deployment	Vercel	Independent application deployment.
Version control	Git	Source-code change tracking.
Repository hosting	GitHub	Remote source-code hosting and collaboration.

6. Datasets Used

The platform primarily handles operational data generated through user interactions and application workflows. It should therefore be described as an operational-data-driven application unless a separate external dataset or model-training dataset was integrated.

6.1 Device Registration Data

- Device ID
- Device name, category, brand and model
- Physical condition and damage description
- Power, screen and battery condition
- Device weight or other measurements, where collected

6.2 Transfer Data

- Transfer ID and device ID
- Source and destination application
- Transfer status
- Transfer timestamp
- Supporting file references

6.3 Assessment Data

- Assessment ID
- Repairability score
- Circular recovery score
- Estimated repair cost
- Component recovery value
- Estimated refurbished value
- Recommended action
- Estimated CO_2 avoided

6.4 Supporting Files

- Device photographs or images, where supported
- Diagnostic reports
- PDF documents
- Other permitted device-related attachments

Dataset note: These categories describe records handled by the application. They are not, by themselves, evidence of a public dataset used to train a machine-learning model. If a public dataset or benchmark was used, add its exact name, source URL, version, fields and preprocessing steps here.

7. Prompts Used for Website Construction

AI-assisted development tools were used to support application construction, UI refinement, API integration, deployment preparation and debugging. The following are representative prompt summaries based on the development workflow; they are not claimed to be a verbatim archive of every prompt.

Prompt 1 – Collection application
Develop a Python/Flask e-waste collection and recovery application with device registration, device details, file uploads, transfer management and a dashboard. Use a modular backend and preserve a consistent professional UI.

Prompt 2 – Refurbishment application
Develop an independent Python application for refurbishment intelligence. Include incoming-device management, assessment recording, repairability and recovery evaluation, cost/value estimates and recommendation management.

Prompt 3 – Frontend dashboard
Create a professional dashboard with clear navigation, device tables, status indicators, analytical summaries and device-management workflows. Preserve the established visual theme and responsive behavior.

Prompt 4 – REST API integration
Connect the two independent Flask applications through REST APIs. Implement device transfer from Collection to Refurbishment and reverse transmission of assessment results using JSON and API-key authentication.

Prompt 5 – Bidirectional file exchange
Implement secure file exchange using shared cloud storage. Transfer file metadata and storage references through the API and ensure the receiving application can access the files securely.

Prompt 6 – Database integration
Configure both applications to use Supabase PostgreSQL with separate collection and refurbishment schemas. Implement SQLAlchemy models, connection handling and controlled migrations.

Prompt 7 – Production deployment
Prepare both Flask applications for independent Vercel deployment. Configure production environment variables, database connections, API URLs, authentication credentials and cloud storage integration.

Prompt 8 – Integration testing
Test device registration, transfer acknowledgement, file references, assessment generation and reverse callback processing. Validate authentication, HTTP responses, database persistence and error handling.

8. API Integration

REST APIs connect the two independent applications. JSON is used as the structured interchange format. The following routes reflect the integration routes identified during project development; confirm the deployed route definitions and exact request schema against the final code before submission.

8.1 Collection to Refurbishment

Method and endpoint: POST /api/v1/refurbishment/devices

Purpose: Transfers device information and supporting file references from the Collection application to the Refurbishment application.

Illustrative JSON request:

```
{
 "device_id": "EW-2026-D86F5C8F",
 "transfer_id": "TRF-2026-D86F5C8F",
 "device": {
  "id": "EW-2026-D86F5C8F",
  "name": "HP Omen Hyper X",
  "type": "Laptop",
  "brand": "HP",
  "model": "Omen Hyper X",
  "condition": "Good"
 },
 "files": []
}
```

8.2 Refurbishment to Collection

Method and endpoint: POST /api/v1/assessment-results

Purpose: Sends assessment information from the Refurbishment application to the Collection application.

Illustrative JSON request:

```
{
 "device_id": "EW-2026-D86F5C8F",
 "transfer_id": "TRF-2026-D86F5C8F",
 "assessment_id": "ASM-2026-D86F5C8F",
 "repairability_score": 88,
 "circular_recovery_score": 84,
 "repair_cost": 3200.00,
 "component_recovery_value": 7500.00,
 "estimated_refurbished_value": 18500.00,
 "recommendation": "Refurbish",
 "co2_avoided_kg": 24.5
}
```

The JSON examples are illustrative. The exact field names, required fields, accepted value ranges and response bodies must match the implemented API contracts.

8.3 API Security

- Separate credentials are used for the two communication directions.
- Credentials are configured through server-side environment variables.
- Secrets should not be hardcoded in source code or exposed in browser-side JavaScript.
- Protected endpoints validate the configured API credential before processing requests.

8.4 Other API Functions

- Device retrieval
- Transfer tracking
- Dashboard data retrieval
- Assessment management
- Assessment-result delivery
- Application health checks

9. Users, Inputs and Outputs

9.1 Users

A. Collection Operator

The collection operator manages electronic devices entering the e-waste workflow.

- Register devices and enter specifications.
- Record physical and operational condition.
- Upload supporting documents where supported.
- Initiate and monitor transfers.
- View assessment results returned by the refurbishment application.

B. Refurbishment Operator

The refurbishment operator manages incoming devices and records their assessment and recovery information.

- View incoming devices.
- Review device details and supporting information.
- Record assessment values.
- Estimate repair costs and component recovery values.
- Select or record a recommended recovery action.
- Send assessment results to the collection application.

9.2 Input Types and Formats

Input category	Input type / format	Application
Device details	Text / string	Collection
Device category	Selection / string	Collection
Device condition	Selection or text / string	Collection
Device weight	Numeric / integer or decimal	Collection
Device images	Image files; exact accepted	Collection

	formats depend on implementation	
Diagnostic documents	PDF or other configured file types	Collection
Transfer information	Structured JSON	Both
Repairability score	Numeric	Refurbishment
Circular recovery score	Numeric	Refurbishment
Repair cost	Numeric / decimal	Refurbishment
Component recovery value	Numeric / decimal	Refurbishment
Recommendation	Categorical text / string	Refurbishment
Assessment result	Structured JSON	Both

The actual accepted file formats, maximum upload size and required fields are determined by the deployed application's validation and configuration.

9.3 Outputs

Collection Application
- Registered device records
- Device and transfer tracking information
- Transfer status and supporting file references
- Assessment results received from Refurbishment
- Device recovery information

Refurbishment Application
- Incoming device records
- Assessment records
- Repairability and recovery scores
- Estimated repair cost and component recovery value
- Refurbishment or parts-recovery recommendation
- Assessment delivery acknowledgement

Integrated Platform
- Bidirectional data exchange
- Transfer acknowledgements
- Persistent device and assessment records
- Shared file references
- Device lifecycle information

10. References

1. Baldé, C. P., et al. (2024). The Global E-waste Monitor 2024. United Nations Institute for Training and Research (UNITAR) and International Telecommunication Union (ITU). https://ewastemonitor.info/the-global-e-waste-monitor-2024/

2. Ministry of Environment, Forest and Climate Change, Government of India. (2022). E-Waste (Management) Rules, 2022. G.S.R. 801(E), 2 November 2022. https://moef.gov.in/
3. Central Pollution Control Board (CPCB). E-Waste Management. Government of India. https://cpcb.nic.in/
4. Flask Documentation. Pallets Projects. https://flask.palletsprojects.com/
5. SQLAlchemy Documentation. https://docs.sqlalchemy.org/
6. PostgreSQL Documentation. PostgreSQL Global Development Group. https://www.postgresql.org/docs/
7. Supabase Documentation. https://supabase.com/docs
8. Vercel Documentation. https://vercel.com/docs
9. Git Documentation. https://git-scm.com/doc

Conclusion

The Circular E-Waste Recovery & Refurbishment Intelligence Platform provides a connected digital workflow for electronic-waste collection, device assessment and refurbishment coordination. By connecting two independent Python applications through bidirectional REST API communication, the platform supports structured information exchange, device traceability, assessment-result feedback and shared file references.

The project demonstrates the use of distributed web applications, RESTful integration, managed relational databases and cloud deployment to support a more organized and recovery-oriented electronic-waste management process.